

Inteligencia Artificial en Dispositivos de Borde: una aplicación en IoT Industrial

Ernesto Miranda, Silvia Ruiz

Facultad de Ciencias de la Alimentación y Facultad de Ciencias de la Administración,
Universidad Nacional de Entre Ríos
3200, Concordia
{ernesto.miranda, silvia.ruiz}@uner.edu.ar

Resumen. La inteligencia artificial (IA) en dispositivos de borde permite la optimización de sistemas del Internet de las Cosas (IoT) al realizar localmente el procesamiento de datos reduciendo la latencia y preservando la privacidad. Esto permite a los dispositivos IoT operar de manera autónoma, incluso en entornos con conectividad limitada. Los modelos de IA ligera, como redes neuronales optimizadas, facilitan la implementación de aplicaciones en tiempo real, como monitoreo industrial, ciudades inteligentes y salud digital. Sin embargo, enfrenta desafíos, como la limitación de recursos y la necesidad de algoritmos eficientes. Este trabajo explora tendencias, ventajas y desafíos de la integración entre IA e IoT en dispositivos de borde, presentando una prueba de concepto aplicada al ámbito industrial demostrando la viabilidad de la implementación en microcontroladores sin una pérdida de precisión significativa.

Palabras clave: Inteligencia artificial, Dispositivos de borde, microcontroladores, Internet de la cosas

1 Introducción

El número de dispositivos IoT (Internet of Things) se ha incrementado sustancialmente en los últimos años. Se estima que, para este año, habrá aproximadamente 21.500 millones de dispositivos conectados a Internet [1], lo que equivale a un promedio de 2,6 dispositivos por persona a nivel mundial, a pesar de que amplias regiones del planeta aún carecen de acceso a la red.

Estos dispositivos generalmente tienen un poder de cómputo limitado y una memoria pequeña, pero sin embargo pueden generar enormes cantidades de datos recolectadas por sus sensores. Los dispositivos IoT son empleados en diversos ámbitos y aplicaciones como ser el monitoreo industrial (Industrial IoT o IIoT), domótica y salud electrónica (e-Health), entre otros.

Hasta el momento, el desarrollo de la computación en la nube (Cloud Computing) respaldó el crecimiento de IoT almacenando y procesando los datos generados por estos dispositivos. Sin embargo, este enfoque comienza a mostrar falencias que lo vuelven impráctico para algunas aplicaciones. Problemas como la latencia en la

comunicación, el consumo del ancho de banda y la privacidad [2] están llevando el procesamiento lo más cerca de la fuente posible, incorporando IA en los dispositivos.

De esta forma surge el concepto de computación en el borde (Edge Computing), el cual consiste en el procesamiento de la información en su lugar de origen y transmitir a servidores remotos únicamente la información necesaria para el procesamiento que requiera otras capacidades de cómputo [3].

Edge Computing también tiene sus desafíos, dado que no es simple ejecutar modelos de IA en los microcontroladores sobre los que están contruidos la mayoría de los dispositivos IoT dadas las limitaciones ya mencionadas. Sin embargo, recientes avances en esta materia están permitiendo la ejecución de modelos simples, o con una mínima complejidad, en dispositivos limitados en recursos de hardware.

En este sentido, el presente trabajo tiene como objetivo implementar un dispositivo de borde capaz de detectar anomalías en las mediciones de temperatura de equipos industriales, con el fin de alertar sobre posibles fallas en estos. Para ello, se establece un flujo de trabajo completo que abarca desde la construcción y entrenamiento del modelo de machine learning (ML), su optimización e implementación en un microcontrolador, así como la fabricación del prototipo, el desarrollo del software embebido y su integración con IoT. Este enfoque busca demostrar la viabilidad de aplicar IA ligera en entornos industriales con recursos limitados.

2 Estado del Arte

Esta sección tiene como objetivo presentar una visión general de las tendencias actuales, los desafíos pendientes y las oportunidades futuras en el campo de la IA en dispositivos de borde. Se enfoca en cinco pilares fundamentales: los algoritmos de ML utilizados en dispositivos de borde, las optimizaciones de los modelos de IA, el uso de diferentes arquitecturas de soluciones, las características del hardware, así como también las comunicaciones y protocolos empleados.

2.1 Algoritmos de Machine Learning

A los fines de implementar los modelos de IA en los dispositivos de borde varios algoritmos pueden emplearse, siendo los más populares los de Support Vector Machine (SVM), Deep Learning (DL) y Redes Neuronales.

De todos los algoritmos mencionados, las Redes Neuronales se destacan porque su principal demanda de poder computacional ocurre durante la fase de entrenamiento. En cambio, durante la fase de inferencia, el consumo de recursos computacionales es mucho menor pudiendo emplearse sin problemas en hardware de pocos recursos.

Para la implementación de los algoritmos de ML, existen muchos frameworks, de los cuales los más populares son PyTorch [4] y TensorFlow [5]. Ambos ofrecen herramientas para ejecutar modelos de IA en dispositivos de borde. Dado que la opción que ofrece PyTorch [6], es un proyecto aún en etapas tempranas de desarrollo,

se seleccionó la opción ofrecida por TensorFlow, llamada TensorFlow Lite (TFLite) [7] para este trabajo por su madurez y su amplio soporte de hardware.

2.3 Optimizaciones de los Modelos de Inteligencia Artificial

El diseño de modelos de IA para su ejecución en dispositivos de borde se centra en la reducción del número de parámetros, con el objetivo de minimizar el uso de memoria y disminuir el tiempo de inferencia, procurando mantener una alta precisión.

Esto se logra mediante técnicas de compresión, que permiten adaptar los modelos a las restricciones de memoria y procesamiento de los dispositivos de borde. Entre las técnicas más empleadas se encuentran la cuantificación y la poda [8]. Mientras que la cuantificación (quantization) reduce la precisión de los pesos del modelo, la poda (pruning) consiste en eliminar conexiones innecesarias dentro de la red neuronal.

2.4 Arquitectura de las soluciones

Según los requerimientos específicos, pueden emplearse distintas arquitecturas [9], las cuales se agrupan en tres categorías principales.

Por un lado se encuentra la computación en el dispositivo (edge computing), donde los modelos de IA se ejecutan únicamente en el dispositivo IoT. Por otro lado están las arquitecturas basadas en servidores de borde (fog computing), donde la información se envía a servidores cercanos capaces de ejecutar modelos más complejos. Y por último se encuentra la arquitectura de computación conjunta, en la cual se combina la posibilidad de ejecutar parte de los modelos de IA en el dispositivo de borde, en los servidores de borde y en los servidores en la nube (cloud computing).

En la Figura 1 se observan las tres arquitecturas mencionadas anteriormente y cómo la latencia en la respuesta aumenta a medida que se emplean más capas.

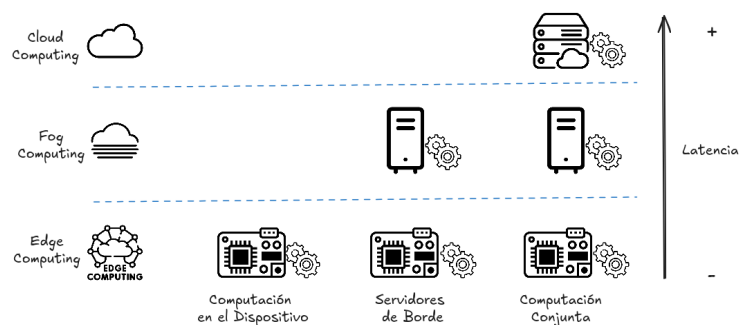


Fig. 1. Arquitectura de las aplicaciones

2.5 Hardware

La correcta elección del hardware es determinante para soportar la ejecución de los algoritmos y modelos de IA. Al respecto, existe una amplia variedad de arquitecturas, las cuales poseen capacidades de procesamiento y memoria muy diferentes. Las métricas utilizadas para seleccionar el hardware adecuado estarán basadas en una combinación de: precisión, consumo de energía, productividad/latencia y costo [10].

En el sitio Web de Tensorflow Lite para Microcontroladores [11] se mencionan una serie de dispositivos compatibles, de los cuales se seleccionaron algunos con el fin de comparar sus especificaciones más relevantes:

Tabla 1. Especificaciones técnicas de algunos microcontroladores.

Dispositivo	Frecuencia CPU	Memoria Flash	Memoria RAM	Almacenamiento
Arduino Nano 33 BLE S.	64 Mhz	1 MB	256 KB	-
STM32F746 Dev Kit	216 Mhz	1 MB	340 KB	16 MB
Adafruit EdgeBadge	120 Mhz	512 KB	192 KB	2 MB
Espressif ESP-32 DevKit C	160 Mhz	320 KB	512 KB	4-8 MB
Raspberry Pi 4	1.8 Ghz	-	1-8 GB	

2.6 Comunicaciones y protocolos

Una de las características de las placas de desarrollo empleadas en el ámbito de IoT es la integración de tecnologías de comunicación. En Tabla 2, se mencionan algunas de las tecnologías de comunicación empleadas en IoT.

Tabla 2. Tecnologías de comunicación empleadas en IoT.

Tecnología	Velocidad	Distancia
LoRa	0,3 a 50 kbps	5-10 km
Zigbee	250 kbps	10 - 100 m
Bluetooth LE	1 Mbps	50 m
Bluetooth 5	2 Mbps	200 m
WiFi 802.11n	600 Mbps	250 m
4G/LTE	150 Mbps	15 km
5G	10-50 Gbps	2 km

El protocolo de comunicaciones más empleado en este ámbito es el MQTT (Message Queuing Telemetry Transport) [12] el cual se caracteriza por ser eficiente, consumir pocos recursos y estar diseñado específicamente para dispositivos con recursos limitados y redes de bajo ancho de banda. Este protocolo utiliza un modelo de comunicación publicador/suscriptor, donde los dispositivos publican mensajes en topics (temas) y otros dispositivos se suscriben a esos topics para recibir los mensajes. Este modelo desacopla a los publicadores de los suscriptores, por medio de un broker central, facilitando la escalabilidad y la flexibilidad en las redes IoT.

3 Objetivo de la investigación

Todo equipo industrial estará sujeto a una serie de condiciones ambientales ideales para su correcto funcionamiento. Entre estas condiciones estarán determinados un rango de temperatura ambiente, de humedad, la altitud y otras condiciones atmosféricas como ser la presencia de ciertos gases o contaminantes.

Existen diversas normas como la IEC 60034-1 [13], la NEMA MG-1 [14] y la NBR-17094 [15] que establecen rangos de temperatura y humedad ambientales para las condiciones normales de servicio. La operación del dispositivo por fuera de estos rangos puede provocar una reducción en su vida útil, por lo que resulta imprescindible su monitoreo.

Las variaciones en las condiciones ambientales podrían traer aparejadas variaciones en las condiciones de trabajo del dispositivo. Mientras estas últimas no superen los márgenes recomendados, el dispositivo podría continuar operando sin inconvenientes. Pero una variación en la temperatura de operación no relacionada con cambios ambientales podría indicar un desperfecto en el dispositivo.

El presente trabajo propone construir y entrenar un modelo de comportamiento recopilando información sobre las condiciones ambientales y de trabajo del dispositivo. Dicho modelo será capaz de predecir cuál debería ser la temperatura de operación del dispositivo para las condiciones ambientales del momento. Cualquier desvío, por fuera de un umbral, entre la temperatura real de operación y la temperatura predicha por el modelo será considerada como una anomalía posiblemente provocada por un desperfecto en el dispositivo.

4 Metodologías y Técnicas

En esta sección se describen los pasos para la construcción del prototipo, la creación y entrenamiento del modelo de TensorFlow, su conversión a TensorFlow Lite y su integración en el programa del microcontrolador.

4.1 Construcción del prototipo

Se construyó un prototipo que fuera capaz de capturar datos de las condiciones ambientales y las condiciones de operación del dispositivo industrial. Para esto se empleó un microcontrolador ESP-32 con capacidad de interconexión por WiFi y Bluetooth y demás especificaciones ya brindadas en la Tabla 1.

El esquema de conexiones y los sensores empleados pueden observarse en la Figura 2.

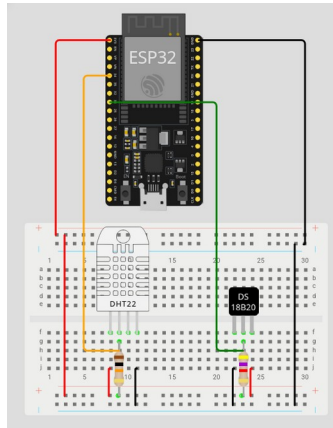


Fig 2. Esquemático de conexiones del microcontrolador con los sensores.

Este prototipo puede emplearse tanto en la etapa de captura de datos con los que entrenar el modelo como luego, con el modelo ya entrenado para realizar las inferencias, detectar y comunicar las anomalías.

4.2 Construcción y entrenamiento del modelo de TensorFlow

Para la construcción del modelo de TensorFlow se obtuvieron datos sobre las condiciones ambientales y de operación del dispositivo industrial durante 30 días lo que constituyó un dataset de 720 muestras. Se realizó una limpieza del dataset y se tomó el 80% del mismo como datos de entrenamiento y el 20% restante como datos de prueba.

Se realizó un análisis estadístico de las características de los datos de entrenamiento y se realizó la normalización de los valores a los fines de facilitar el entrenamiento y hacer que el modelo resultante no dependa de la elección de las unidades utilizadas en la entrada.

Se empleó una red neuronal Feed-Forward (o perceptrón multicapa) utilizando Sequential de Keras. Este modelo es el adecuado para resolver problemas de regresión, donde el objetivo es predecir un valor continuo. La primera capa (capa de entrada) toma el vector de entrada que contiene los datos de temperatura y humedad actuales. La última capa (capa de salida) consta de una única neurona, la cual produce la salida final del modelo con un único valor continuo el cual corresponderá a la temperatura interna a la cual el dispositivo debería estar operando.

Como función de pérdida se empleó una métrica de uso común para los problemas de regresión como es el error cuadrático medio (MSE). Las métricas utilizadas para evaluar el rendimiento del modelo en las etapas de entrenamiento y validación fueron el error absoluto medio (MAE) y el MSE.

Los resultados obtenidos con los datos de prueba fueron aceptables y pueden observarse en la Figura 3.

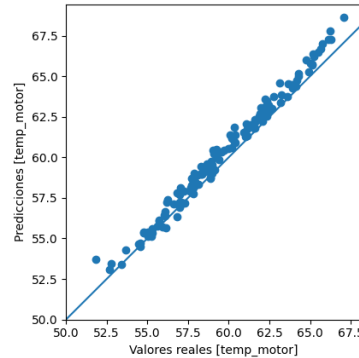


Fig. 3. Resultados obtenidos utilizando los datos de prueba.

Finalmente, el modelo se almacenó en un archivo, obteniendo un tamaño de aproximadamente 61 KB. Aunque este tamaño permitiría su almacenamiento en el microcontrolador utilizado, es importante considerar la implementación de optimizaciones adicionales para adaptarse a modelos más complejos en el futuro.

Se utilizó TFLiteConverter para convertir el modelo de TensorFlow a Tensorflow Lite logrando una reducción a un 32 % del tamaño original. Para la primera optimización se seleccionó la técnica Quantization, obteniendo una reducción al 13% del tamaño original del modelo. Por último se empleó la técnica experimental Sparsify logrando resultados similares a los obtenidos con la conversión a TensorFlow Lite.

Para cada uno de los modelos convertidos y optimizados se realizaron pruebas para determinar su precisión sin que se evidencien pérdidas de precisión significativas.

4.3 Programación del microcontrolador

Para poder utilizar TensorFlow Lite en el microcontrolador se empleó la biblioteca EloquentTinyML, la cual proporciona una interfaz fácil de usar para cargar modelos de aprendizaje automático optimizados para microcontroladores y realizar inferencias directamente en estos dispositivos.

En la Figura 4 se puede observar el flujo de ejecución del bucle principal del programa.

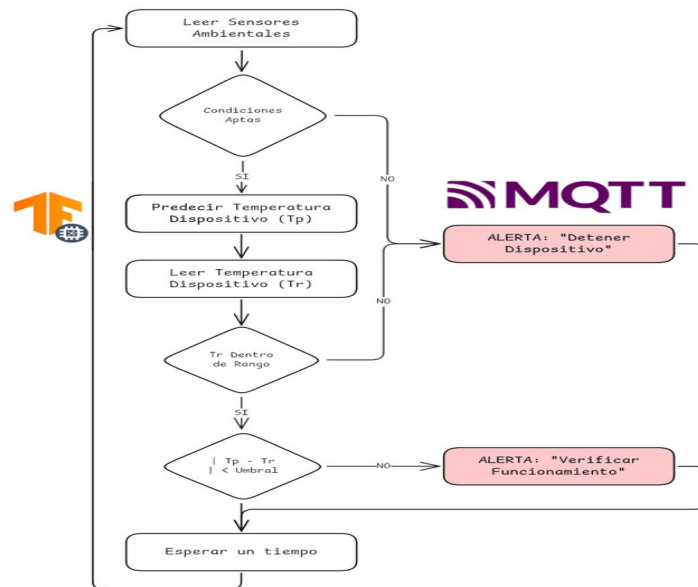


Fig. 4. Diagrama de flujo del programa del microcontrolador.

4.4 Comunicaciones

Dadas las características de la placa empleada en la construcción del prototipo, la tecnología empleada para la comunicación fue WiFi 802.11b/g/n el cual proporciona un ancho de banda más que suficiente para la aplicación.

El protocolo de comunicación empleado para enviar las alertas es MQTT, ya descrito anteriormente. En la Figura 5 se muestran los tópicos publicados, los dispositivos involucrados y los tópicos a los que se suscriben.

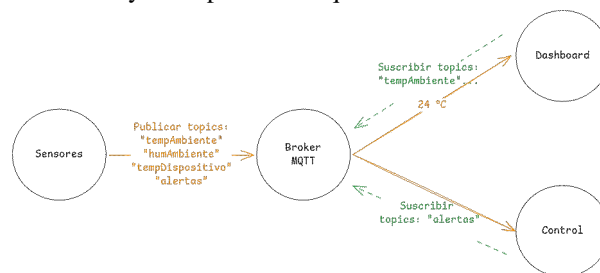


Fig. 5. Funcionamiento del protocolo MQTT.

A los fines de visualizar la información se construyó un panel de control empleando Node-RED [16]. En el panel, el cual se observa en la Figura 6, es posible visualizar las alertas, monitorear las condiciones ambientales y del dispositivo industrial y contar con un registro histórico de las mismas.

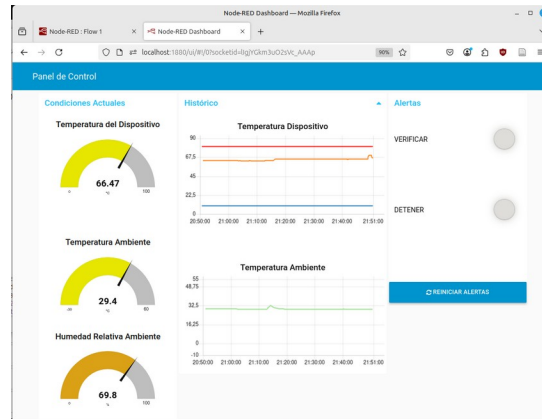


Fig. 6. Panel de control construido con Node-Red.

5 Resultados

Una de las principales preocupaciones al convertir modelos de TensorFlow en TensorFlow Lite es la posible pérdida de precisión. Para este caso no se evidenció pérdida de precisión alguna en los modelos convertidos a TensorFlow Lite, ni en aquellos a los que se les aplicaron diversas optimizaciones tal como se muestra en la Tabla 3.

Tabla 3. Métricas de precisión y tamaño del modelo para las diferentes optimizaciones.

	Modelo			
	TensorFlow		TF Lite	
Optimización	-	-	Sparsify	Quantization
MAE	0,3736	0,3736	0,3736	0,3712
MSE	0,225	0,2249	0,2249	0,2222
Tamaño (en Bytes)	61.544 B	19.548 B	19.620 B	8.056 B

Si bien no se realizaron comparativas de los tiempos de inferencia entre las optimizaciones, los mismos fueron más que aceptables, encontrándose en el orden de los 250 a 290 microsegundos como se observa en la Figura 7.

```
(Tiempo de inferencia = 258 microseg.)
Temperatura = 29.50 °C Humedad = 69.80 % Dispositivo = 67.16 °C Predicción 64.88 °C
(Tiempo de inferencia = 290 microseg.)
Temperatura = 29.40 °C Humedad = 69.70 % Dispositivo = 66.79 °C Predicción 64.79 °C
(Tiempo de inferencia = 253 microseg.)
Temperatura = 29.40 °C Humedad = 69.80 % Dispositivo = 66.47 °C Predicción 64.79 °C
(Tiempo de inferencia = 268 microseg.)
```

Fig. 7. Salida del monitor serial mostrando información de los sensores y tiempo de inferencia.

6 Conclusiones

Como conclusión se demostró que resulta posible implementar un modelo de TensorFlow Lite en un microcontrolador sin pérdida de precisión y empleando un tiempo de inferencia más que aceptable.

Como trabajo futuro, sería recomendable incorporar mejoras al prototipo que permitan detener en forma automática la operación del dispositivo controlado al detectar una anomalía.

Este trabajo también plantea varios interrogantes que abren la puerta a futuras líneas de investigación, tales como la integración de más puntos de medición en el dispositivo industrial o la inclusión de variables ambientales adicionales para mejorar la precisión del modelo. No obstante, siempre se deberá considerar que estas modificaciones no comprometan la capacidad de ejecución del modelo en el hardware empleado, dada sus limitaciones.

7 Referencias

1. Internet de las cosas (IoT): dispositivos conectados en el mundo 2015-2030, <https://es.statista.com/estadisticas/517654/prevision-de-la-evolucion-de-los-dispositivos-conectados-para-el-internet-de-las-cosas-en-el-mundo/>, último acceso 07/2025.
2. Merenda, M., Porcaro, C., Iero, D.: Edge Machine Learning for AI-Enabled IoT Devices: A Review. *Sensors*. 20, 2533 (2020).
3. Singh, R., Gill, S.S.: Edge AI: A survey. *Internet Things Cyber-Phys. Syst.* 3, 71-92 (2023).
4. PyTorch, <https://pytorch.org/>, último acceso 07/2025.
5. TensorFlow, <https://www.tensorflow.org/?hl=es-419>, último acceso 07/2025.
6. PyTorch ExecuTorch, <https://pytorch.org/executorch-overview>, último acceso 08/2025.
7. TensorFlow Lite, <https://www.tensorflow.org/lite/guide?hl=es-419>, último acceso 05/2025.
8. Han, S., Mao, H., Dally, W.J.: Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding, <http://arxiv.org/abs/1510.00149>, (2016).
9. Kuchuk, H., Malokhvii, E.: Integration of IoT with cloud, fog, and edge computing: a review. *Adv. Inf. Syst.* 8, 65-78 (2024).
10. Sze, V., Chen, Y.-H., Emer, J., Suleiman, A., Zhang, Z.: Hardware for Machine Learning: Challenges and Opportunities.
11. TensorFlow Lite para microcontroladores, <https://www.tensorflow.org/lite/microcontrollers?hl=es-419>, último acceso 07/2025.
12. MQTT - The Standard for IoT Messaging, <https://mqtt.org/>, último acceso 07/2025.
13. IEC 60034: Rotating electrical machines, (2024).
14. NEMA MG1: Motors and Generators, <https://law.resource.org/pub/us/cfr/ibr/005/nema.mg-1.2009.pdf>, (2009).
15. ABNT NBR 17094: Máquinas elétricas girantes, (2018).
16. Low-code programming for event-driven applications: Node-RED, <https://nodered.org/>, último acceso 07/2025.